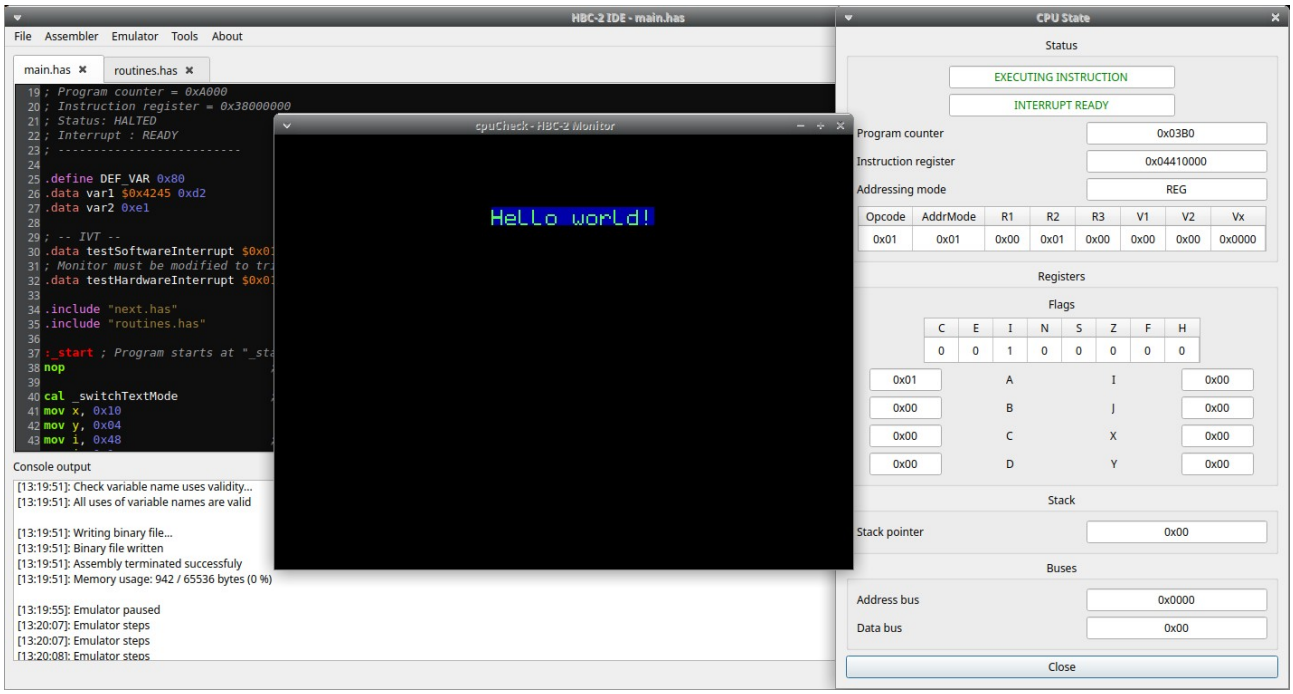
HBC-2 IDE
Instruction Manual



Summary

I. Generalities

II. Instruction format and addressing modes

III. Random Access Memory and I/O interfaces

IV. Hardware interrupts

V. Software interrupts

VI. Instruction set

Generalities

The HomeBrew Computer 2 (or HBC-2) is a home made 8 bit CPU designed to understand the basics of computer science via a simplified central processing unit.

Specifications

Data bus width	8 bits
Address bus width	16 bits
RAM maximum capacity	64 kB
Clock frequency	2 Mhz
Instruction size	1 – 4 bytes
Instruction set size	54
Architecture	Von Neumann / RISC
Stack size	16-bit stack pointer (STK), grows downward STK = 0xFE00 at reset (first push at 0xFDFF) *(no under/overflow flag)*
I/O ports	256 *(port #0 reserved for CPU <> IOD communication)*

An IDE (Integrated Development Environment) is available to program this CPU in a custom assembly language, and emulate the computer's behavior.

This software allows for a deep insight of the internal mechanisms of the computer. Various windows offer a view of the RAM in real-time, of the CPU registers, and many other elements.

The goal of this manual is to provide detailed explanation about the computer's systems, the IDE and the assembly language that has been developed.

Instruction format

and addressing modes

Instructions



Instructions are 8-32 bits (or 1-4 bytes) long, divided in up to 4 sections :

1. A 6 bits operand (64 instructions maximum, 48 implemented),
2. A 4 bits addressing mode (16 addressing modes maximum, 9 implemented),
3. Two 3 bits register selectors (8 registers, A, B, C, D, I, J, X, Y),
4. Two 8 bits user-defined values.

The CPU first fetches **one** byte, giving it the operand.
→ Some instruction use no addressing mode, thus using only 1 byte.

If necessary to check for the addressing mode, the CPU fetches a **second** byte.
→ Some instruction use only 2 selected registers, which are available in the second byte. Thus taking no more space.

The addressing mode may require **1 or 2** additional bytes.

Bad opcode and bad addressing modes

If the CPU finds an **invalid opcode**:
→ It treats it as the **nop** instruction, and skips no further bytes.

If the CPU finds a **valid opcode** with **invalid addressing mode**:
→ It clear the interrupt flag and halts (**hlt**), thus freezing the computer until reset.

Addressing modes (A.M.)

Many instructions can work in multiple addressing modes. Those modes allow to use pointers, mandatory for a Turing-complete machine.

The instruction set described at the end of this manual gives information about which addressing modes you can use for each instruction.

None	0x0
Reg	0x1
Reg / Imm8	0x2
Reg / Ram	0x3
RamReg / ImmReg / Offset	0x4
Stack / Offset	0x5
Reg16	0x6
Imm16	0x7
Imm8	0x8

None

No data is specified, the operand is sufficient.



Reg

Two 8-bit values retrieved from registers 1 and 2.



Reg / Imm8

One 8-bit value retrieved from register 1 and one user-defined 8-bit value.



Reg / Ram

One user-defined 16-bit value and one 8-bit value retrieved from register 1.



RamReg / ImmReg / Offset

Register 1 and 2 are concatenated as a 16-bit address to retrieve one 8-bit value from memory.

Register 1 is the most significant byte (**MSB**).

V2 is a user-defined 8-bit value that is added to the address as an offset.

The 8-bit value in register 3 is used directly.



Stack / Offset

One 8-bit value read from memory at the stack pointer plus an offset (V1).

Register 1 holds the data.



Reg16

Register 1 and 2 are concatenated in a 16-bit value used directly.

Register 1 is the most significant byte (MSB).



Imm16

One user-defined 16-bit value.



Imm8

One user-defined 8-bit value.



Random Access Memory (RAM)

and I/O interfaces

The microprocessor uses two buses to access data from external devices, one for **data**, and one for the **address**. Those buses are used both to access RAM and I/O devices. Accessing I/O devices is done via the I/O Driver chip, **IOD**, which is specifically design for this CPU, and handles both I/O accesses and I/O interrupts.

Data bus width	8 bits
Address bus width (RAM)	16 bits *(64 kB)*
Address bus width (I/O)	8 bits *(256 ports)*

This means that when accessing **I/O ports**, the CPU only uses the **8 least significant bits** of the address bus. Whatever is on the 8 other bits is left unused. The port number is read by the IOD chip, which is triggered by the IE pin (I/O Enable).

Control lines

RAM	RW *(Read / Write)* + RE *(Ram Enable)*
I/O access	RW *(Read / Write)* + IE *(I/O Enable)*

To access data from **RAM**, the CPU sets the R/W pin accordingly, and sets the **RE** pin to **HIGH**.
To access data from **I/O**, the CPU sets the R/W pin accordingly, and sets the **IE** pin to **HIGH**.

When the RE pin is set to HIGH, the IE pin is always set to LOW, and vice versa.

Hardware interrupts

To handle interrupts, the microprocessor implements two control lines, **INT** (pending interrupt) and **INR** (interrupt ready), that can be trigerred by the CPU and the IOD chip.

CPU side

The CPU is not necessarily ready to handle interrupts. An **interrupt flag** is implemented in the flag register. When this flag is **set**, the CPU is capable of handling interrupts. Two instructions control this behaviour : **STI** (Set Interrupt flag), and **CLI** (Clear Interrupt flag).

When the CPU has received an interrupt, and is ready to manage it, it retrieves the I/O port number via the IOD chip, and then jumps to a memory location defined by the programmer in the **interrupt vector table (IVT)**.

The interrupt vector table is a **512 bytes** memory block in the RAM, located from address **0xFE00** to address **0xFFFF**. Those address holds pointers that the CPU uses to set the **PC** (Program Counter register). As a reminder, RAM addresses are 2 bytes long, and there are 256 I/O ports that are numbered from 0x00 to 0xFF.

I/O port number	Address in the IVT
0x00 *(reserved for CPU <> IOD communication)*	0xFE00 << 0xFE01
0x01	0xFE02 << 0xFE03
[...]	*[...]*
0xFE	0xFFFC << 0xFFFD
0xFF	0xFFFE << 0xFFFF

Procedure :

1. The IOD chip sets the **INT** control line to **HIGH**,
2. If the interrupt flag is set to **HIGH**, the CPU finishes the current instruction and sets the **INR** control line to **HIGH** for one clock cycle,
3. The IOD chip then sets **INT** control line to LOW, and sends the I/O port number on the **address bus** for **one clock cycle**, as well as the data from the device on the **data bus**,
4. The CPU then reads the I/O port number on the address bus, pushes the **I register**, pushes the **PC register** to the **stack**, stores the I/O data in the **I register**, sets the **interrupt flag** to **LOW**, sets the **INR** control line to **LOW**, and then jumps to the memory location pointed by the **interrupt vector table**.
5. On the next clock cycle, if the interrupts queue is not empty, the IOD chip set the **INT** control line back to **HIGH** until the CPU is ready to manage it (step 2).
6. When the CPU has finished processing the interrupt (by reaching the IRT instruction), it pops the PC register out of the stack to get back to work on the code that was interrupted, and pops the I register. If INT pin is HIGH, it manages the next interrupt immediately.

IOD chip side

The CPU does not always have enough time to process an interrupt before the next one is trigerred. Therefore, the IOD chip implements a 256 interrupts queue via a FIFO queue.

If an interrupt is trigerred but the previous one was not processed by the CPU yet, then the I/O port number is stacked in the queue, and the IOD chip will keep the **INT** control line **HIGH** until the CPU is ready to manage it.

If the queue is full, any new interrupt will be discarded.

Software interrupts

The CPU implements the **INT** instruction, that triggers a software interrupt on the I/O port number defined by the programmer in the **V1** value of the instruction.

This kind of interrupt does not end up in a dialog between the CPU and IOD chip. Also, they have no priority on hardware interrupts.

__Register I__ will be pushed onto the stack, and the CPU will handle this interrupt just a regular hardware interrupt. Therefore, instruction __INT__ sets the data bus with the content of register I.

Instruction set

Abreviation	Operand	A.M.	Description	Flags
NOP	0x00		No operation	
ADC	0x01	*Reg*	R1 = R1 + R2 + Carry flag	**C N Z**
		Reg / Imm8	R1 = R1 + V1 + Carry flag	
		Reg / Ram	R1 = R1 + $(Vx) + Carry flag	
		RamReg / ImmReg / Offset	R3 = R3 + $((R1 << R2) + V2) + Carry flag	
		Stack / Offset	R1 = R1 + $(STK + V1) + Carry flag	
ADD	0x02	*Reg*	R1 = R1 + R2	
		Reg / Imm8	R1 = R1 + V1	
		Reg / Ram	R1 = R1 + $(Vx)	
		RamReg / ImmReg / Offset	R3 = R3 + $((R1 << R2) + V2)	
		Stack / Offset	R1 = R1 + $(STK + V1)	
AND	0x03	*Reg*	R1 = R1 & R2	**Z N**
		Reg / Imm8	R1 = R1 & V1	
		Reg / Ram	R1 = R1 & $(Vx)	
		RamReg / ImmReg / Offset	R3 = R3 & $((R1 << R2) + V2)	
		Stack / Offset	R1 = R1 & $(STK + V1)	
CAL	0x04	*Reg16*	STK = STK − 1, $(STK) = PC_LSB STK = STK − 1, $(STK) = PC_MSB [1]PC = R1 << R2	
		Imm16	STK = STK − 1, $(STK) = PC_LSB STK = STK − 1, $(STK) = PC_MSB PC = Vx	
CLC	0x05		Clears the Carry flag	**C**

1 PC: Program Counter

CLE	0x06		Clears the Equal flag	**E**
CLI	0x07		Clears the Interrupt flag	**I**
CLN	0x08		Clears the Negative flag	**N**
CLS	0x09		Clears the Superior flag	**S**
CLZ	0x0A		Clears the Zero flag	**Z**
CLF	0x0B		Clears the Inferior flag	**F**
CMP	0x0C	*Reg*	[2]FR = R1 ? R2	**S E F**
		Reg / Imm8	FR = R1 ? V1	
		RamReg / ImmReg / Offset	FR = R3 ? $((R1 << R2) + V2)	
		Stack / Offset	FR = R1 ? $(STK + V1)	
DEC	0x0D	*Reg*	R1 = R1 - 1	**C N Z**
		Imm16	$(Vx) = $(Vx) - 1	
		RamReg / ImmReg / Offset	$((R1 << R2) + V2) = $((R1 << R2) + V2) - 1	
HLT	0x0E		Halts the CPU (if **I** flag is not set, the computer freezes until reset)	**H**
IN	0x0F	*Reg*	Data bus = R1 Address bus = R2 IOD Receive pin = HIGH	
OUT	0x10		Data bus = R2 Address bus = R1 IOD Send pin = HIGH	
INC	0x11	*Reg*	R1 = R1 + 1	**C N Z**
		Imm16	$(Vx) = $(Vx) + 1	
		RamReg / ImmReg / Offset	$((R1 << R2) + V2) = $((R1 << R2) + V2) + 1	
INT	0x12	*Imm8*	Software interrupt with port number V1: STK = STK − 1, $(STK) = I STK = STK − 1, $(STK) = PC_LSB STK = STK − 1, $(STK) = PC_MSB PC = $(0xFE00 + 2 × V1) Clears Interrupt flag	**I**

2 FR: Flag Register

Abreviation	Operand	A.M.	Description	Flags
IRT	0x13		[3]PC_MSB = $([4]STK), STK = STK + 1 [5]PC_LSB = $(STK), STK = STK + 1 I = $(STK), STK = STK + 1 Sets Interrupt flag	**I**
JMC	0x14	*Reg16*	PC = $(R1 << R2) if Carry flag	
		Imm16	PC = $(Vx) if Carry flag	
JME	0x15	*Reg16*	PC = $(R1 << R2) if Equality flag	
		Imm16	PC = $(Vx) if Equality flag	
JMN	0x16	*Reg16*	PC = $(R1 << R2) if Negative flag	
		Imm16	PC = $(Vx) if Negative flag	
JMP	0x17	*Reg16*	PC = $(R1 << R2)	
		Imm16	PC = $(Vx)	
JMS	0x18	*Reg16*	PC = $(R1 << R2) if Superior flag	
		Imm16	PC = $(Vx) if Superior flag	
JMZ	0x19	*Reg16*	PC = $(R1 << R2) if Zero flag	
		Imm16	PC = $(Vx) if Zero flag	
JMF	0x1A	*Reg16*	PC = $(R1 << R2) if Inferior flag	
		Imm16	PC = $(Vx) if Inferior flag	
STR	0x1B	*RamReg / ImmReg / Offset*	$((R1 << R2) + V2) = R3	
		Reg / Ram	$(Vx) = R1	
		Stack / Offset	$(STK + V1) = R1	
LOD	0x1C	*RamReg / ImmReg / Offset*	R3 = $((R1 << R2) + V2)	
		Reg / Ram	R1 = $(Vx)	
		Stack / Offset	R1 = $(STK + V1)	
MOV	0x1D	*Reg*	R1 = R2	
		Reg / Imm8	R1 = V1	
NOT	0x1E	*Reg*	R1 = ! R1	**Z N**
		Imm16	$(Vx) = ! $(Vx)	

3 PC_MSB: Program Counter most significant byte
4 STK: Stack pointer
5 PC_LSB: Program Counter least significant byte

Abreviation	Operand	A.M.	Description	Flags
OR	0x1F	*Reg*	R1 = R1 | R2	
		Reg / Imm8	R1 = R1 | V1	
		Reg / Ram	R1 = R1 | $(Vx)	
		RamReg / ImmReg / Offset	R3 = R3 | $((R1 << R2) + V2)	
		Stack / Offset	R1 = R1 | $(STK + V1)	
POP	0x20	*Reg*	R1 = $(STK) STK = STK + 1	
PSH	0x21		STK = STK - 1 $(STK) = R1	
RET	0x22		PC_MSB = $(STK), STK = STK + 1 PC_LSB = $(STK), STK = STK + 1	
SHL	0x23	*Reg*	Left shift on R1	**C Z N**
ASR	0x24		Arithmetical shift on R1	
SHR	0x25		Right shift on R1	**[6]Z N**
STC	0x26		Sets the Carry flag	**C**
STE	0x27		Sets the Equality flag	**E**
STI	0x28		Sets the Interrupt flag (after next instruction is done)	**I**
STN	0x29		Sets the Negative flag	**N**
STS	0x2A		Sets the Superior flag	**S**
STZ	0x2B		Sets the Zero flag	**Z**
STF	0x2C		Sets the Inferior flag	**F**
PSF	0x2D		STK = STK - 1, $(STK) = FR	[7]
POF	0x2E		FR = $(STK), STK = STK + 1	**All**

6 **SHR** is equivalent to **ASR**, but the bit shifted to the right is discarded, instead of being carried over. Therefore the carry flag is not raised.

7 **POF** overwrites all flags, Interrupt and Halt included (popping a value with the Halt flag set halts the CPU). In an interrupt handler, **PSF** first and **POF** just before **IRT** is safe: both flags are cleared when the handler starts.

Abreviation	Operand	A.M.	Description	Flags
SUB	0x2F	*Reg*	R1 = R1 - R2	**C N Z**
		Reg / Imm8	R1 = R1 - V1	
		Reg / Ram	R1 = R1 - $(Vx)	
		RamReg / ImmReg / Offset	R3 = R3 - $((R1 << R2) + V2)	
		Stack / Offset	R1 = R1 – $(STK + V1)	
SBB	0x30	*Reg*	R1 = R1 - R2 - Carry flag	**C N Z**
		Reg / Imm8	R1 = R1 - V1 - Carry flag	
		Reg / Ram	R1 = R1 - $(Vx) - Carry flag	
		RamReg / ImmReg / Offset	R3 = R3 - $((R1 << R2) + V2) - Carry flag	
		Stack / Offset	R1 = R1 – $(STK + V1) - Carry flag	
XOR	0x31	*Reg*	R1 = R1 ^ R2	**Z N**
		Reg / Imm8	R1 = R1 ^ V1	
		Reg / Ram	R1 = R1 ^ $(Vx)	
		RamReg / ImmReg / Offset	R3 = R3 ^ $((R1 << R2) + V2)	
		Stack / Offset	R1 = R1 ^ $(STK + V1)	
ASP	0x32	*Imm8*	STK = STK + V1	
		Imm16	STK = STK + Vx	
[8]SSP	0x33	*Imm8*	STK = STK - V1	
		Imm16	STK = STK - Vx	
[9]RSP	0x34	*Reg16*	R1 = STK_MSB, R2 = STK_LSB	
[10]WSP	0x35	*Reg16*	STK = R1 << R2	
		Imm16	STK = Vx	

8 **SSP** reserves space for local variables when a function starts, and **ASP** frees it before **RET**. The Imm16 mode is for large local arrays (more than 255 bytes).
9 **RSP** copies the stack pointer into a register pair, for the address of a local (&n).
10 **WSP** sets the stack pointer, to place the stack elsewhere than its reset position.